

Desenvolvimento de Funcionalidades CRUD (Delete e Update)

Objetivo da Aula

Compreender o conceito de CRUD e criar serviço REST com persistência no MySQL via Sequelize, englobando as operações de alteração e exclusão.

Apresentação

Enquanto, nos contextos mais comuns, a consulta e a inserção de dados não apresentam grande risco para o sistema, a alteração e a exclusão acabam trazendo maior impacto. Devemos ser cuidadosos ao definir as restrições que serão aplicadas ao conjunto que será modificado ou removido, pois corremos o risco de impactar toda a tabela, e não apenas os registros necessários.

Como estamos lidando com Sequelize, precisamos nos aprofundar no uso de operadores para a cláusula `where` para que possamos delimitar o conjunto de registros que será consultado, modificado ou removido. Veremos como esses operadores são flexíveis, permitindo a construção de regras bastante complexas.

Após estudar o uso dos operadores, iremos aplicá-los para a construção dos métodos de alteração e exclusão, ao nível do DAO, efetuando o controle pela API REST. Finalmente, vamos acrescentar ao cliente Axios as chamadas para os novos serviços disponibilizados, completando as funcionalidades de um CRUD na arquitetura MVC, dividido em Back-end e Front-end.

1. Opções da Cláusula Where

Ao trabalhar com **Sequelize**, a cláusula **where** corresponde às restrições de um comando **SQL**, tanto para consulta quanto para operações de alteração ou exclusão. Porém, a sintaxe do Sequelize é baseada em **JSON**, deixando-a mais próxima da que é utilizada por bancos documentais, como o MongoDB.

A cláusula **where** pode ser utilizada nos comandos **destroy**, **update**, **findAll** e **findOne**, entre outros. Em qualquer situação, ele deverá restringir a ação do comando, através do fornecimento das condições para seleção.

No fragmento seguinte é feita uma pequena comparação entre o comando SQL e a sintaxe do Sequelize.

SQL: `Select nome from personagens where ataque > 1000;`

Sequelize: `db.Personagem.findAll({where:{ataque:[Op.gt]:1000}},
attributes:['nome']});`

Observe como no SQL os operadores ficam entre os operandos, como o sinal de maior, enquanto no Sequelize, por utilizar JSON, costuma colocar o operador antes dos operandos. Observe a comparação a seguir, para uma seleção com uso do operador lógico **or**.

SQL: `Select * from personagens where ataque > 1000 or
defesa < 500;`

Sequelize: `db.Personagem.findAll({where:{
[Op.or]: [ataque:[Op.gt]:1000},
defesa:[Op.lt]:500]}}});`

Note como no Sequelize temos a utilização do operador **or** antes de um vetor JSON, com os operandos utilizando outros operadores. Para uma grande cadeia de operações do tipo **or**, seria um formato prático, pois todas as condições do vetor serão combinadas pelo operador.



Destaque

Na ausência de um operador para relacionar várias condições, será utilizado o operador **and**, indicando que todas as condições devem ser satisfeitas para selecionar o registro.

Também temos o operador **like**, muito utilizado no SQL, disponível para o Sequelize. Ele facilita a pesquisa por padrões de nomes na base de dados.

```
SQL: Select * from personagens where nome like 'A%';
```

```
Sequelize: db.Personagem.findAll({where:  
    {nome:{[Op.like]: 'A%'}}});
```

A consulta retornaria todos os personagens com nome começado com "A", da mesma forma que ocorreria no SQL. Temos ainda os operadores **notLike**, **regexp** e **between**, que representam, respectivamente, a negação do like, uso de expressões regulares e valor entre dois limites.

```
SQL: Select * from personagens where nome like 'A%';
```

```
Sequelize: db.Personagem.findAll({where:  
    {nome:{[Op.like]: 'A%'}}});
```

Tanto o SQL quanto o Sequelize irão retornar os personagens com ataque entre 100 e 200, com os limites inclusos. O mesmo comando poderia ser escrito com os operadores **gte** e **lte**, como no exemplo seguinte.

```
db.Personagem.findAll({where: {ataque:  
    {[Op.gte]:100, [Op.lte]:200}}});
```

Através do operador **not** é possível inverter todo um conjunto de operações, e vetores com utilização direta são similares ao operador **in**.

```
SQL: Select * from personagens where not  
    (id in (1,2,3) and ataque < 200);
```

```
Sequelize: db.Personagem.findAll({where: {[Op.not]:[  
    {id:[1,2,3]},  
    {ataque:{[Op.lt]:200}}]}]);
```

Outra operação muito comum é o teste de nulidade de um campo.

```
SQL: Select * from personagens where nome is null;
```

```
Sequelize: db.Personagem.findAll({where: {nome:  
    {[Op.is]:null}}});
```

Existem muitos outros operadores disponíveis no Sequelize, permitindo que regras complexas sejam aplicadas nas restrições de consulta.

2. Operações Delete e Update no DAO

Voltando ao nosso projeto do servidor REST para personagens, atualmente temos as operações de consulta e inclusão implementadas. Precisamos ainda das operações de alteração e exclusão para completar o CRUD.

Lembrando que as quatro operações básicas sobre os registros são:

- **Create**, referindo-se à inclusão de um registro;
- **Read**, para a recuperação dos dados de uma tabela;
- **Update**, quando o objetivo é alterar os dados atuais do registro; e
- **Delete**, para a remoção do registro.

Para dar continuidade à implementação de nossa **API de personagens**, precisamos acrescentar a alteração e a exclusão dos personagens. A seguir, são replicados os comandos anteriores para configuração das entidades.

```
npm init -y  
npm install sequelize mysql2  
npm install --save-dev sequelize-cli  
npx sequelize-cli init  
npx sequelize-cli model:generate --name Personagem  
    --attributes nome:string,ataque:integer,defesa:integer
```

Através da sequência de comandos, teríamos a configuração de um projeto **NodeJS**, acréscimo das bibliotecas para o **MySQL** e o **Sequelize**, incluindo a interface **sequelize-cli**.

Em seguida, o sequelize-cli é utilizado para configurar a persistência no projeto e gerar a entidade Personagem.

Após executar a sequência de comandos, é necessário configurar a conexão com o banco de dados no arquivo **config.json**.

```
"development": {  
  "username": "root", "password": "admin123",  
  "database": "jogo_dev", "host": "127.0.0.1",  
  "dialect": "mysql"  
},
```

Finalmente, o sequelize-cli deve ser utilizado para criar o banco de dados e a tabela de personagens, utilizando os comandos da listagem seguinte.

```
npx sequelize-cli db:create  
npx sequelize-cli db:migrate
```

Para implementar a remoção da entidade via Sequelize, vamos acrescentar o método **delete** em **PersonagemDAO**, no arquivo **personagem-dao.js**.

```
const db = require('../models/index.js');  
  
class PersonagemDAO{  
  // Outros métodos permanecem inalterados  
  
  delete = async(_id) => {  
    return await db.Personagem.destroy(  
      {where: {id: _id}});  
  }  
}  
  
module.exports = PersonagemDAO
```

Como podemos observar, temos uma chamada para o método **destroy**, do Sequelize, com uma restrição **where** sobre a **chave primária**. Isso vai garantir que apenas o registro solicitado seja removido. Uma implementação alternativa da remoção pode ser observada a seguir.

```
const p1 = await db.Personagem.findById(_id);  
p1.destroy();
```

Na segunda forma, a entidade é encontrada via `findById`, e o método `destroy` é solicitado a partir dela, garantindo também a remoção do registro correto. Agora vamos implementar a alteração da entidade, acrescentando o método **update** em **PersonagemDAO**, no arquivo **personagem-dao.js**.

```
const db = require('../models/index.js');  
  
class PersonagemDAO{  
    // Outros métodos permanecem inalterados  
  
    update = async(_id, objJSON) => {  
        return await db.Personagem.update(objJSON,  
            {where: {id: _id}});  
    }  
}  
  
module.exports = PersonagemDAO
```

Temos um código muito parecido com o da remoção, com a restrição **where** sobre a **chave primária**, mas agora também temos as informações que serão alteradas no registro, fornecidas em **objJSON**. Para a alteração é utilizado o método **update** do Sequelize. Também temos uma implementação alternativa para a alteração, com base no método **save**, que persiste o estado atual da entidade no banco de dados.

```
const p1 = await db.Personagem.findByPk(_id);  
p1.nome = objJSON.nome;  
p1.ataque = objJSON.ataque;  
p1.defesa = objJSON.defesa;  
p1.save();
```

Na forma alternativa, a entidade é encontrada via `findByPk`, cada atributo do personagem é alimentado por um dos campos de `objJSON`, e o método `save` é invocado, a partir da entidade, armazenando os dados na tabela.

3. Exclusão e Alteração via REST

Com as novas funcionalidades adicionadas em nosso DAO, precisamos que nosso servidor reconheça as rotas necessárias para acioná-las. Nosso trabalho será concentrado apenas no **tratamento das rotas** para alteração e exclusão. Lembrando aqui que estamos utilizando um servidor **Express**, com suporte a **CORS** e utilização de **Body Parser**.

```
npm install express body-parser cors
```

Vamos acrescentar os tratamentos necessários em **jogo-rotas.js**, conforme a listagem apresentada a seguir.

```
const express = require('express')  
const PersonagemDAO = require('../daos/personagem-dao.js')  
const router = express.Router();  
const dao = new PersonagemDAO();  
  
router.delete('/:id', async(req,res)=>{  
    let dados = await dao.delete(req.params.id);  
    res.json(dados);  
})
```

```
router.put('/:id', async(req,res)=>{
    let dados = await dao.update(req.params.id, req.body);
    res.json(dados);
})

// As demais funções de tratamento não são alteradas

module.exports = router
```

Acrescentamos duas funções de tratamento para o **endpoint** parametrizado pelo **id**, uma para as rotas com método **PUT** e outra para o **DELETE**. Em ambos os casos, o **id** é recuperado da rota por meio de **req.params.id**.

Para o método **delete** do roteador, chamamos o método de mesmo nome no DAO, com a passagem do **id**. Isso causará a remoção do personagem, cujo **id** é o fornecido, na base de dados.

Já para o método **put**, além do **id**, utilizamos os dados enviados no corpo da requisição, recuperados em **req.body**, e acionamos o método **update** do DAO. Com a execução do tratamento, os dados atuais da entidade serão substituídos pelos dados que foram enviados na requisição.

Finalizados os acréscimos, temos um Web Service RESTful completo, com todas as operações do CRUD. O código do servidor, que utiliza nosso roteador, é replicado a seguir.

```
const express = require('express');
const bodyParser = require('body-parser');
const cors = require('cors')
const rotaJogo = require('./rotas/jogo-rotas');
const app = express();
app.use(bodyParser.json());
app.use(cors({origin: '*'}));
app.use('/personagens', rotaJogo);
app.listen(3000, () => console.log("Servidor pronto"));
```


Não podemos deixar de observar a configuração de **CORS** aberta, para que possamos acessar pelo cliente **Axios** sem bloqueios. Também é importante que observemos o uso do nosso roteador no caminho de base **personagens**, o que leva ao endereço inicial **http://localhost:3000/personagens**.



Para Praticar

Utilize uma ferramenta como **Postman** ou **Insomnia** para testar sua API REST completa. Verifique também os dados no MySQL, através do **MySQL Workbench**.

4. Cliente para Exclusão e Alteração com Axios

Agora vamos completar nosso cliente, para incluir as opções de exclusão e alteração de personagens. O cliente é baseado em Axios e a única configuração necessária é a instalação da biblioteca em um projeto NodeJS.

```
npm init -y
npm install axios
```

O cliente de comunicação, no arquivo **cliente.js**, deverá ser alterado para o código que é apresentado a seguir. As chamadas já existentes foram alteradas para uma sintaxe mais compacta.

```
const axios = require('axios')
const api = axios.create({baseUrl: "http://localhost:3000"});

class Cliente {
  incluir = async(nome,ataque,defesa) =>
    await(await api.post(
      'personagens', {nome,ataque,defesa})).data;
  obterTodos = async () =>
    await(await api.get('personagens')).data;
```

```
obter = async (id) =>
    await(await api.get(`personagens/${id}`)).data;

// Novos métodos a partir daqui...
alterar = async(id,nome,ataque,defesa) => {
    let resp = await api.put(`personagens/${id}`,
        {nome,ataque,defesa});
    return await resp.data;
}
excluir = async(id) => {
    let resp = await api.delete(`personagens/${id}`);
    return await resp.data;
}
}

module.exports = Cliente
```

Com relação aos métodos adicionados, **excluir** é mais simples, e faz apenas uma chamada via **delete**, com uma rota formada dinamicamente pelo caminho de base seguido do **id**. Já para o método **alterar**, utilizamos **put** e novamente a rota com o **id** ao final, mas agora com os novos dados para o personagem sendo incluídos no **corpo** da requisição.

Em ambos os casos, assim como nos demais métodos de **Cliente**, o corpo da resposta é retornado ao final, por meio do campo **data**. Com relação ao arquivo **index.js**, devemos completar as funções que faltam, relacionadas à alteração e exclusão de personagens, conforme a listagem que é apresentada a seguir.

```
const excluir = async () => {
    const idStr = await ler("Id para excluir:");
    await cliente.excluir(eval(idStr));
}
```

```
const alterar = async () => {
    const id = eval(await ler("Id para alterar:"));
    const dados = await cliente.obter(id);
    console.log(dados);
    const [nome, ataque, defesa] = await obterDados();
    await cliente.alterar(id,nome,ataque,defesa);
}
// O restante do código não é alterado
```

A função **excluir**, da interface, apenas solicita o **id** para exclusão, converte para o formato numérico e invoca o método **excluir** no cliente de comunicação, gerando uma requisição para nossa API via método **DELETE** do HTTP.

Com relação à função **alterar**, temos inicialmente a solicitação do **id**, para que seja feita uma busca via método **obter**, e a impressão dos valores atuais na tela. Em seguida, os novos valores são requisitados, via função **obterDados**, e o método **alterar** é invocado, com a passagem dos valores, gerando a requisição via método **PUT** para nossa API, com subsequente alteração dos dados.



Destaque

Quando queremos retornar vários valores simultaneamente a partir de uma função, podemos definir as variáveis em um vetor, como ocorre ao invocarmos o método **obterDados**.

Figura 1: Operação de alteração em andamento

```
nome: 'xpto',
ataque: 150,
defesa: 210,
createdAt: '2023-08-30T09:41:16.000Z',
updatedAt: '2023-08-30T09:41:16.000Z'
}
Nome: Dragao2
```

Fonte: Elaboração própria.

```
const readline = require('readline');
const leitor = readline.createInterface({input: process.stdin,
                                         output: process.stdout});

const ler = (pergunta) => {
  return new Promise(resolve => {
    leitor.question(pergunta, (input) => resolve(input)));
  });
}
```

A biblioteca **readline**, do núcleo do NodeJS, permite receber dados a partir do teclado, mas ela segue o modelo de **callback**, onde o método **question**, do leitor, apresenta a pergunta e retorna a resposta para a callback. Nesse caso, como estamos com processos assíncronos, retornamos o valor digitado por meio de uma **Promise**, a qual aguarda o final da digitação e depois emite o valor. Devido ao fato de que o código da interface com o usuário é muito extenso, ele é replicado a seguir, para melhor entendimento do sistema cliente.

```
const readline = require('readline');
const Cliente = require("./cliente.js")
const cliente = new Cliente();
const leitor = readline.createInterface({input: process.stdin,
                                         output: process.stdout});

const ler = (pergunta) => {
  return new Promise(resolve => {
    leitor.question(pergunta, (input) => resolve(input)));
  });
}

const menu = async () => {
  console.log("1-Listar 2-Incluir 3-Alterar 4-Excluir 0-Sair");
  const valor = await ler("Digite a opcao:"); return eval(valor);
}

const obterDados = async() => {
  const nomeStr = await ler("Nome:");
}
```

```
    const ataqueStr = await ler("Ataque:");
    const defesaStr = await ler("Defesa:");
    return [nomeStr, eval(ataqueStr), eval(defesaStr)];
}

const incluir = async () => {
    const [nome, ataque, defesa] = await obterDados();
    await cliente.incluir(nome, ataque, defesa);
}

const alterar = async () => {
    const id = eval(await ler("Id para alterar:"));
    const dados = await cliente.obter(id); console.log(dados);
    const [nome, ataque, defesa] = await obterDados();
    await cliente.alterar(id, nome, ataque, defesa);
}

const excluir = async () => {
    const idStr = await ler("Id para excluir:");
    await cliente.excluir(eval(idStr));
}

const listar = async () => {
    const dados = await cliente.obterTodos();
    for(let obj of dados) console.log(obj);
}

const principal = async () => {
    let opcao;
    do {
        opcao = await menu();
        switch(opcao){
            case 1: await listar(); break;
        }
    } while (opcao !== 0);
}
```

```
        case 2: await incluir(); break;
        case 3: await alterar(); break;
        case 4: await excluir(); break;
    }
} while (opcao != 0);
}
```

```
principal().then(()=>leitor.close());
```

Considerações Finais da Aula

Aqui, pudemos observar algumas utilizações do operador de restrições Op, do Sequelize. Vimos o quanto ele é flexível e robusto, dando suporte às mais diversas restrições utilizadas no SQL, mas indo além, já que lida com dados no formato JSON, o qual apresenta características hierárquicas que não fazem parte do modelo relacional.

Durante a aplicação dos operadores nas restrições, ao nível da classe DAO, vimos também que as modificações mais simples, restritas a uma única entidade, podem ser feitas a partir de suas próprias instâncias, após uma busca com base na chave primária. Essa é uma abordagem com menor risco de impacto sobre a base, porém é menos eficiente, por executa mais comando SQL em background.

Com nossa classe DAO completa, pudemos implementar a resposta para as requisições DELETE e PUT na API REST, e consequente criação das chamadas via Axios na interface do cliente. Ao final, obtivemos um CRUD completo, usando a arquitetura MVC, com uma interface de usuário muito simples, através da linha de comando, mas que permite definir uma estratégia de implementação para testes de ponta-a-ponta em qualquer sistema com uso de APIs REST.

Material Complementar



Sequelize – Operators

Sequelize.

O artigo, parte da documentação do Sequelize, fornece a descrição e exemplos para cada operador disponível via cláusulas Op.

Link para acesso: <https://sequelize.org/docs/v7/querying/operators/> (acesso em: 14 set. 2023.)

Referências

FLANAGAN, D. **JavaScript: o guia definitivo**. Porto Alegre: Bookman, 2014. ISBN 9788565837484. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92164?guid=1691771378051&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691771378051%26quantidadePaginas%3d1%26codigoRegistro%3d92164%2392164&i=8>. Acesso em: 14 set. 2023.

OLIVEIRA, Cláudio Luís V.; ZANETTI, Humberto Augusto P. **JavaScript descomplicado**: programação para a Web, IoT e dispositivos móveis. São Paulo: Érica, 2020. ISBN 9788536533100. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92165?guid=1691773379732&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691773379732%26quantidadePaginas%3d1%26codigoRegistro%3d92165%2392165&i=24>. Acesso em: 14 set. 2023.

OLIVEIRA, Cláudio Luís V.; ZANETTI, Humberto Augusto P. **Node.js**: programe de forma rápida e prática. São Paulo: Expressa, 2021. ISBN 9786558110217. Disponível em: <https://biblioteca.sophia.com.br/terminal/9564/acervo/detalhe/92479?guid=1691770593236&returnUrl=%2fterminal%2f9564%2fresultado%2flistar%3fguid%3d1691770593236%26quantidadePaginas%3d1%26codigoRegistro%3d92479%2392479&i=4>. Acesso em: 14 set. 2023.